

ayer 6:28 p.m.

<https://www.youtube.com/watch?v=caa2Um8EgJQ>

esto es un proyecto completamente aparte, para diseñar una plataforma y/o sistema de tickets para el depto de soporte técnico (TI) de una empresa. para ello revisa el video adjunto, y propón una idea de resolver esta de manera simple y sencilla. en donde al mandante solo se le cobrará una implementación y mensualmente los ajustes en caso de requerirlo. la empresa tiene un total de 185 trabajadores y muchos voluntarios, ya que el interesado en contratar es la fundación techo para chile

Sí. Lo trataría **completamente separado de Córdor 360°**.

Hay una oportunidad bastante buena acá, pero haría una corrección importante al enfoque: **no construiría inicialmente “un sistema de tickets” complejo**, sino una **mesa de ayuda TI muy liviana**, pensada específicamente para una organización con ~185 trabajadores y una cantidad de voluntarios que puede ser mucho mayor.

Intenté abrir directamente el video, pero YouTube no permitió recuperar su contenido desde aquí. Por eso no voy a inventar lo que aparece en él. Si me adjuntas el video, una transcripción o capturas, puedo hacer después el análisis exacto del flujo mostrado.

Lo que sí podemos definir desde ya

TECHO-Chile tiene una estructura particularmente adecuada para este tipo de solución: trabaja con equipos permanentes y una base muy amplia de voluntariado; actualmente declara presencia en 11 regiones y más de 60 mil voluntarios/as que se han sumado a su trabajo.  TECHO C... +1

Además, TECHO ya tiene iniciativas digitales propias, como **MiTECHO**, que centraliza actividades, inscripciones, perfiles y comunicaciones de voluntariado.  TECHO La...

Eso significa que **yo no intentaría competir ni reemplazar sus sistemas existentes**. El proyecto debería limitarse al problema concreto:

“Cuando alguien de **TECHO** tiene un problema tecnológico, ¿cómo lo informa, quién lo recibe, cómo se prioriza, quién lo resuelve y cómo sabemos que efectivamente quedó resuelto?”

Mi propuesta: una Mesa de Ayuda TI simple

La arquitectura podría ser:

Usuario → Portal de Soporte → Ticket → Mesa TI → Técnico → Resolución → Cierre

Y nada más en la primera versión.

1. Portal de solicitud

Una página web extremadamente sencilla:

“Soporte TI — TECHO Chile”

Con:

- Iniciar sesión con Google/Microsoft.
- Crear solicitud.
- Ver mis solicitudes.
- Revisar estado.
- Agregar comentarios.
- Adjuntar archivos, imágenes o capturas.
- Recibir notificaciones.

El usuario no debería tener que entender conceptos técnicos.

En vez de:

“Selecione categoría ITIL”

mostrar:

¿Qué problema tienes?

-  Computador
-  Correo electrónico
-  Contraseña / acceso
-  Internet / WiFi
-  Impresora

-  Teléfono
 -  Aplicación / sistema
 -  Cuenta nueva / permisos
 -  Otro
-

2. Ticket automático

Al enviar:

TK-000152

Problema: No puedo acceder a mi correo

Solicitante: Juan Pérez

Área: Administración

Ubicación: Santiago

Prioridad: Normal

Estado: Nuevo

Y desde ese momento todo queda registrado.

3. Panel del equipo TI

Un dashboard sencillo.

Pendientes

Ticket	Problema	Usuario	Prioridad	Estado
TK-152	Correo	Juan	Alta	Nuevo
TK-151	Notebook	María	Media	En proceso
TK-150	Impresora	Pedro	Baja	Esperando usuario

Con filtros:

Nuevos | En proceso | Esperando usuario | Resueltos | Cerrados

Eso probablemente ya resolvería una parte enorme del problema.

4. SLA muy básico

No implementaría inicialmente una metodología ITIL gigantesca.

Bastaría con:

Crítico

Sistema caído / problema que afecta a muchas personas.

Atención: inmediata.

Alto

Usuario no puede trabajar.

Atención: < 4 horas.

Normal

Problema que permite continuar trabajando.

Atención: < 1 día hábil.

Bajo

Solicitud, configuración o mejora.

Atención: 2–3 días.

Esto además permite que el jefe de TI pueda demostrar posteriormente:

“Tenemos 94% de tickets resueltos dentro del SLA.”

5. Una función que sí agregaría desde el principio

Base de conocimiento

Cada vez que TI resuelva un problema recurrente, puede convertirlo en una solución.

Por ejemplo:

¿Cómo configurar el correo institucional en Outlook?

¿Cómo recuperar contraseña?

¿Cómo conectarse a la VPN?

¿Cómo configurar impresora?

¿Cómo solicitar acceso a Google Drive?

Entonces, cuando alguien crea un ticket, el sistema puede buscar:

“Encontramos estas soluciones que podrían ayudarte.”

Y quizás el usuario resuelve el problema **sin intervención humana**.

Esto empieza a convertir la mesa de ayuda en algo mucho más valioso.

6. Y aquí está la parte interesante: los voluntarios

Yo **no metería inicialmente a todos los voluntarios dentro del mismo sistema como si fueran trabajadores permanentes**.

Crearía dos perfiles:

Trabajador

Acceso completo al sistema de soporte.

Voluntario

Acceso simplificado.

Por ejemplo:

Necesito ayuda

Nombre

Correo

Teléfono

Región

Tipo de problema

Descripción

Foto/captura

Y listo.

Eso permite que el sistema pueda manejar cientos o miles de personas sin convertirlo en una pesadilla administrativa.

TECHO declara que su voluntariado puede participar según disponibilidad e interés y que existe una amplia estructura de voluntariado permanente y actividades puntuales.

Por eso el diseño debe ser **escalable en usuarios, pero no necesariamente complejo en funcionalidades**.

Tecnología

Para este proyecto yo evitaría desarrollar todo desde cero.

Una arquitectura económica podría ser:

Frontend

→ aplicación web responsive

Backend

→ Supabase/PostgreSQL

Autenticación

→ Google/Microsoft

Automatización

→ n8n

Notificaciones

→ correo inicialmente

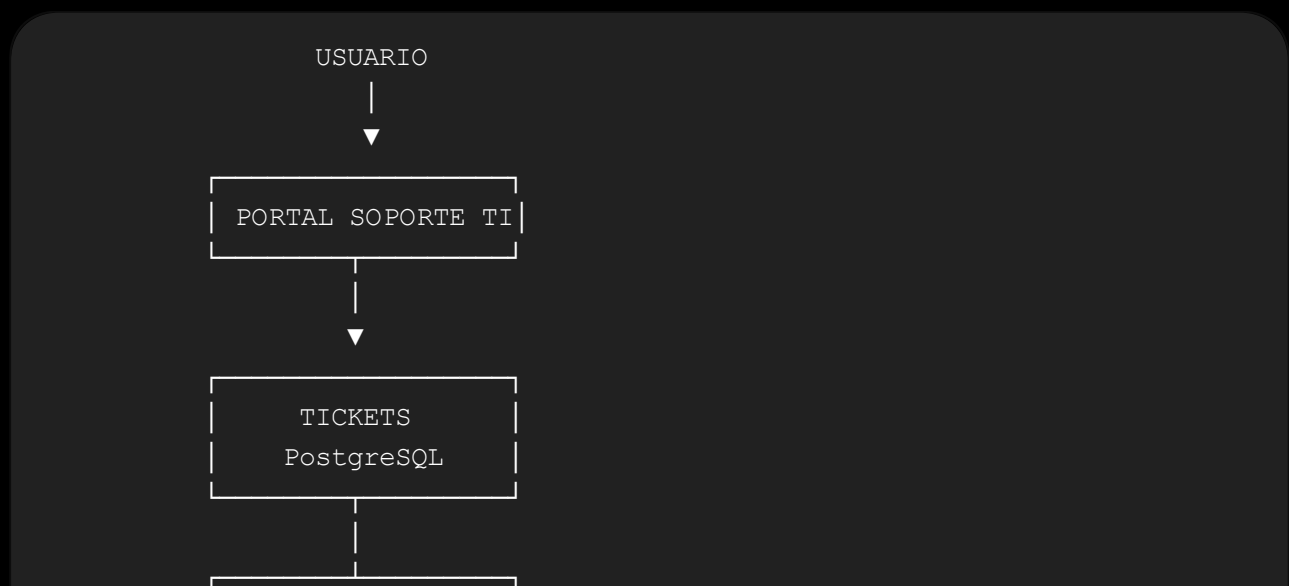
Dashboard

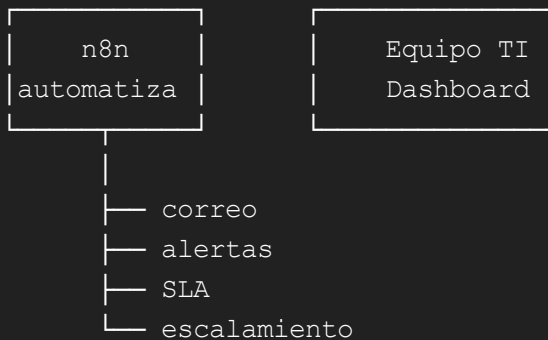
→ aplicación propia / Metabase / Supabase

Archivos

→ almacenamiento tipo S3/Supabase Storage

Esquema





Y comercialmente cambiaría completamente el modelo

Acá está, a mi juicio, la parte más interesante de tu propuesta.

No vendería:

“Desarrollo de software.”

Vendería:

Implementación de Mesa de Ayuda TI.

Y después:

Servicio de mantenimiento y evolución mensual opcional.

Modelo

Concepto	Cobro
Levantamiento y diseño	incluido
Implementación	pago único
Configuración	incluido
Migración inicial	incluido
Capacitación	incluido
Puesta en producción	incluido
Soporte correctivo	mensual opcional
Nuevas funcionalidades	mensual / adicional

Concepto	Cobro
Mejoras y ajustes	mensual
Hosting	costo mensual si corresponde

Esto es mucho más fácil de explicar a una fundación.

Incluso propondría dos modalidades

Opción A — Implementación + mantenimiento

Pago inicial

-

fee mensual bajo

Incluye:

- correcciones
- actualizaciones
- pequeños ajustes
- monitoreo
- respaldo
- soporte del sistema

Opción B — Implementación + pago por evolución

La fundación paga solamente la implementación.

Después:

“Cuando necesitemos modificaciones, solicitamos una cotización.”

Esta opción puede ser especialmente atractiva para TECHO si quiere minimizar costos fijos.

Pero hay algo todavía mejor

No haría que el cliente dependa de ti para absolutamente todo.

La plataforma debería entregarse con:

Administrador TECHO

que pueda:

- crear categorías;
- crear usuarios;
- asignar técnicos;
- modificar prioridades;
- modificar SLA;
- generar reportes;
- administrar respuestas automáticas;
- editar la base de conocimiento.

Así tú vendes **la implementación**, no una dependencia perpetua.

Y posteriormente puedes vender:

“Servicio de evolución de la plataforma.”

MVP que yo le propondría a TECHO

No 30 módulos.

8 componentes.

1. **Login**
2. **Creación de tickets**
3. **Seguimiento**
4. **Panel TI**
5. **Asignación de técnicos**
6. **SLA/prioridades**
7. **Notificaciones**
8. **Reportes**

Y después:

Fase 2

- Base de conocimiento
- Automatizaciones
- Encuesta de satisfacción
- Inventario de activos

- Integración con correo
- WhatsApp/Teams
- IA para clasificación

Fase 3

- IA como primer nivel de soporte
- diagnóstico automático
- sugerencias de solución
- clasificación automática
- detección de problemas recurrentes
- métricas predictivas

La IA no debería estar en el centro inicialmente

Esto es importante.

No vendería a TECHO:

“Una plataforma de soporte TI con inteligencia artificial.”

Eso agrega complejidad y costo.

Vendería:

“Una plataforma simple para ordenar, controlar y medir las solicitudes de soporte TI.”

Después, cuando existan suficientes tickets históricos:

IA → clasificación → sugerencia → autosolución → escalamiento humano.

Eso tiene mucho más sentido.

El producto incluso podría tener nombre propio

Por ejemplo:

Cóndor HelpDesk

o, si quieres mantenerlo completamente independiente:

Nexo TI

MesaTI

Soporte360

HelpDesk Simple

TI Desk

Pero **no lo asociaría a Cóndor 360°**, precisamente porque me indicaste que este proyecto es independiente.

Mi recomendación concreta

Para una organización de **185 trabajadores + una población de voluntarios potencialmente muy superior**, yo apuntaría a un producto que pueda estar operativo en **2–4 semanas**, no en seis meses.

El objetivo de la primera versión debería ser:

“Que ninguna solicitud de soporte TI vuelva a perderse en un correo, WhatsApp, llamada o conversación informal.”

Eso es suficientemente valioso para justificar una implementación y, al mismo tiempo, suficientemente sencillo para que la propuesta económica sea atractiva para una fundación.

Y hay una ventaja adicional: **si funciona bien en TECHO, tienes un producto replicable para otras fundaciones, ONG, colegios, empresas pequeñas y organizaciones con 50–500 usuarios.**

TECHO, además, tiene una estructura distribuida territorialmente y presencia en 11 regiones, lo que hace que una solución centralizada de mesa de ayuda tenga bastante sentido. 

El siguiente paso que haría es analizar exactamente el video que enviaste y convertirlo en un diseño funcional: qué hace el sistema mostrado → qué necesitamos realmente → qué eliminamos por innecesario → arquitectura → módulos → costo de implementación → costo mensual → propuesta comercial para TECHO.

referente a la corrección, no la encuentro que estemos pensando algo diferente, necesito que sea rápido y practico. ya que el depto de soporte esta a cargo solo de una persona.

Adjunto transcripción.

Sí. Con la transcripción, ahora queda mucho más claro, y **coincido contigo en que no estamos planteando cosas diferentes**. Mi corrección anterior fue más de énfasis que de fondo.

De hecho, el video confirma que **la solución que necesitas es bastante más sencilla** y que no tiene sentido convertir esto en un "ITSM enterprise".

El video propone exactamente una lógica pensada incluso para **una persona sola encargándose del soporte**: formulario → ticket → tablero → estados → asignación → fechas → automatizaciones → métricas.

Markdown.md pegado

Lo que yo construiría para TECHO

Pensaría el proyecto así:

Una mesa de ayuda TI web, simple, rápida y personalizada para TECHO, diseñada para que una sola persona pueda administrar todas las solicitudes sin depender de correos, WhatsApp o memoria.

No necesitamos 30 módulos.

Flujo

TRABAJADOR / VOLUNTARIO



FORMULARIO SOPORTE



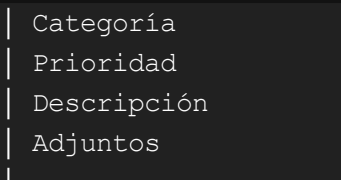
CREACIÓN
DEL TICKET



TICKET #0001

Usuario

Correo



DASHBOARD TI

UNA PERSONA
DE SOPORTE

RESOLUCIÓN

CIERRE

Eso es prácticamente lo que muestra el video.

La clave: no hacer que el usuario “gestione” el ticket

Esto me parece particularmente importante para TECHO.

El usuario solamente debería hacer:

1. Entrar

soporte.techo.cl

2. Completar

¿Cuál es tu problema?

- Computador
- Correo
- Internet
- Impresora
- Software
- Accesos
- Telefonía
- Otro

Prioridad

-  Urgente
-  Alta
-  Normal

- ● Baja

Descripción

Cuéntanos qué problema tienes.

Adjuntar imagen



ENVIAR SOLICITUD

Listo.

El resto lo hace el sistema

Aquí está donde realmente está el valor.

Al crear el ticket:

TK-2026-00125

se registra automáticamente:

- fecha/hora;
- usuario;
- correo;
- categoría;
- prioridad;
- descripción;
- archivo;
- estado;
- tiempo transcurrido.

Y automáticamente:

Estado → ABIERTO

La persona de TI entra a su panel y ve:

● **URGENTES**

1

● **ALTAS**

3

● NORMALES

8

● BAJAS

4

Eso para **una sola persona de soporte** es muchísimo más útil que un sistema sofisticado.

Los estados del video están bien

El video utiliza una secuencia del tipo:

Abierto → **En proceso** → **Información solicitada** → **Cliente respondió** → **Escalado** → **Resuelto**.

Markdown.md pegado

Yo para TECHO incluso la simplificaría un poco:

ABIERTO

Solicitud recibida.

EN PROCESO

La persona de TI está trabajando.

ESPERANDO USUARIO

Se necesita información del solicitante.

ESCALADO

Necesita intervención de un tercero/proveedor.

RESUELTO

Problema solucionado.

CERRADO

Solicitud terminada.

Seis estados. Nada más.

Y el tablero Kanban es perfecto

El propio video muestra el valor de visualizar las solicitudes por estado mediante Board/Kanban.

Markdown.md pegado

Para el responsable de TI:

ABIERTOS	EN PROCESO	ESPERANDO USUARIO	ESCALADOS	RESUELTO
TK-125	TK-121	TK-117	TK-119	TK-110
TK-126	TK-123			TK-111
TK-127				

Y con **drag & drop**:

Abierto → En proceso → Resuelto.

Eso es extremadamente práctico.

La automatización que sí necesitamos

El video muestra una automatización interesante: dependiendo de la categoría, se puede asignar automáticamente a una persona y establecer una fecha.

Markdown.md pegado

Pero en TECHO tenemos una ventaja:

solo hay una persona de soporte.

Por lo tanto:

NO necesitamos asignación automática.

Todos los tickets:

→ **Soporte TI**

Y listo.

Eso simplifica aún más el sistema.

Donde sí pondría automatización

1. Ticket nuevo

Se crea:

TK-000125



Correo al usuario:

Hemos recibido tu solicitud.



Aparece en dashboard.

2. Ticket urgente

Si:

Prioridad = Urgente

entonces:



notificación inmediata.

3. Información solicitada

Si TI cambia:

EN PROCESO → ESPERANDO USUARIO

se manda automáticamente:

“Necesitamos información adicional para continuar con tu solicitud.”

4. Resuelto

Cuando TI marca:

RESUELTO

se envía:

“Tu solicitud ha sido resuelta.”

Y eventualmente:

¿El problema quedó solucionado?

- 👍 Sí
- 👎 No

Y aquí aparece algo muy valioso

Como será una **única persona de soporte**, el dashboard puede convertirse en su verdadero asistente.

Por ejemplo:

HOY

17 tickets

- 2 urgentes
- 4 altos
- 8 normales
- 3 bajos

VENCIDOS

2

ESPERANDO RESPUESTA

3

TIEMPO PROMEDIO DE RESOLUCIÓN

4h 23m

CATEGORÍA MÁS FRECUENTE

Correo electrónico — 31%

Esto es exactamente la clase de información que el video plantea obtener posteriormente mediante registros y dashboard.

Markdown.md pegado

Y yo agregaría una cosa que el video no necesita desarrollar demasiado

Inventario TI

No como módulo gigante.

Simplemente poder relacionar:

Usuario → Equipo

Por ejemplo:

Víctor Pérez

Notebook:

TECHO-NB-023

Marca: Lenovo

Modelo: ThinkPad

Número de serie: XXXXX

Entonces cuando el usuario genere un ticket:

“Mi computador no enciende”

el responsable de TI ya sabe **qué equipo es**.

Esto puede quedar para una segunda etapa si queremos salir rápido.

¿Construirlo desde cero?

Aquí sí haría una decisión diferente respecto del video.

El video está basado en **ClickUp**, con vistas, formularios y automatizaciones.

Markdown.md pegado

Para una demostración interna, perfecto.

Pero si tú quieres **venderle una implementación propia a TECHO**, yo haría:

Frontend

Una web extremadamente sencilla.

Backend

PostgreSQL/Supabase.

Automatización

n8n.

Correo

Google Workspace / SMTP.

Archivos

Storage.

Dashboard

Propio.

Y no dependería de ClickUp como núcleo del producto.

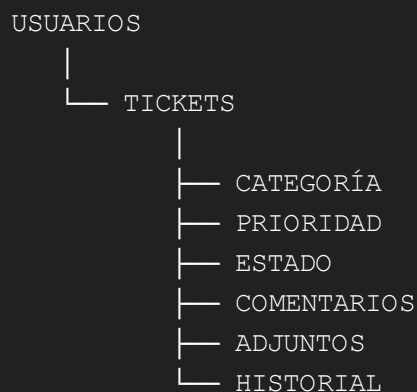
Porque si el modelo comercial es:

implementación + ajustes mensuales,

tú eres dueño de la solución y puedes modificarla.

Pero tampoco construiría un monstruo

La primera versión podría tener solamente estas entidades:



Nada de microservicios.

Nada de arquitectura distribuida.

Nada de IA.

Nada de CRM.

Nada de ERP.

Nada de 20 integraciones.

Una aplicación. Una base de datos. Un flujo.

Modelo comercial

Y acá creo que tienes una propuesta muy vendible.

IMPLEMENTACIÓN

Cobro único por:

- levantamiento;
- diseño;
- desarrollo;
- configuración;
- identidad visual TECHO;
- creación de usuarios;
- configuración de categorías;
- configuración de estados;
- dashboard;
- capacitación;
- puesta en marcha.

MANTENIMIENTO

Mensual opcional:

- correcciones;
- actualizaciones;
- respaldos;
- ajustes;
- cambios menores;
- soporte técnico.

EVOLUCIÓN

Cuando TECHO quiera:

- WhatsApp;
- inventario;
- integración con Google Workspace;
- IA;
- base de conocimiento;
- nuevos dashboards;

- nuevos flujos.

Eso se cotiza aparte o puede entrar en un plan mensual superior.

Mi estimación del MVP

Dado que **solo existe una persona gestionando TI**, yo apuntaría a que el MVP tenga:

Componente	MVP
Portal de solicitudes	✓
Login	✓
Creación ticket	✓
Adjuntos	✓
Categorías	✓
Prioridad	✓
Estados	✓
Dashboard TI	✓
Kanban	✓
Historial	✓
Notificaciones	✓
SLA básico	✓
Estadísticas	✓
Administración	✓
Inventario	⌚ Fase 2
Base conocimiento	⌚ Fase 2
IA	⌚ Fase 3
WhatsApp	⌚ Fase 3

En otras palabras

No estás vendiendo un “software de soporte TI”.

Estás vendiendo algo mucho más fácil de entender:

“Un sistema que centraliza todas las solicitudes de soporte de TECHO y le permite a una sola persona saber qué debe resolver, qué está pendiente y qué ya resolvió.”

Y para **185 trabajadores + voluntarios**, me parece una escala absolutamente razonable para este enfoque.

De hecho, **yo partiría todavía más pequeño**: primero haría que funcione perfectamente para los 185 trabajadores y después habilitaría el mismo formulario para voluntarios. Así no complicamos el lanzamiento.

Si quieres, el siguiente paso lógico es que armemos la especificación exacta del MVP de TECHO —pantalla por pantalla, campos, estados, automatizaciones, base de datos y arquitectura— para después entregársela a Minimax/Claude/Codex y que construyan el prototipo.

Fuentes

correcciones que haría:

- 1.- entrar
- 2.- ID Usuario: email del personal, lo cual indicará quien y el cargo de quien esta solicitando.
- 3.- Incidencia: (las que mencionas en punto 2)
- 4.-Descripción.
- 5.- Adjuntar imagen
- 6.- Enviar.

la prioridad lo da el sistema sacando un calculo entre el cargo, incidencia y descripción. ya que no es lo mismo que

Ver más

Sí. Con esas correcciones, **la lógica queda bastante mejor**. Y especialmente la decisión de que **la prioridad no la determine el usuario**, sino el sistema, me parece correcta.

Eso transforma el proyecto desde un simple formulario de tickets en una **mesa de ayuda inteligente, pero deliberadamente simple**.

1. Frontend: para quien solicita soporte

El usuario prácticamente no tiene que pensar.

Flujo

1. Entrar

Acceso al portal.

2. ID Usuario

El sistema identifica el correo institucional:

usuario@techo.cl

Ese correo está asociado en la base de datos a:

- nombre;
- cargo;
- área;
- ubicación;
- eventualmente región;
- nivel jerárquico.

Por tanto, el usuario **no debería escribir su nombre ni su cargo**.

El sistema ya sabe quién es.

3. Incidencia

Las categorías que definimos:

-  Computador
-  Correo
-  Internet / conectividad
-  Impresora
-  Telefonía
-  Accesos / permisos
-  Software / aplicaciones
-  Cuenta / usuario
-  Otro

Aquí incluso podemos cambiar la palabra "**Incidencia**" por "**Tipo de solicitud**", si queremos que sea más amigable para usuarios no técnicos.

4. Descripción

Campo libre:

Cuéntanos qué necesitas o qué problema tienes.

Aquí es donde el sistema obtiene información para determinar la prioridad.

5. Adjuntar imagen

Opcional.

 Captura de pantalla / fotografía.

Esto puede ser tremendamente útil para que el técnico vea directamente el problema.

6. Enviar

ENVIAR SOLICITUD

Y termina la interacción.

No pediría:

- prioridad;
- SLA;
- fecha;
- técnico;
- estado;
- departamento;
- número de ticket.

Todo eso lo hace el sistema.

2. La prioridad calculada automáticamente

Esta es probablemente la característica más interesante de tu propuesta.

No queremos:

"¿Qué tan urgente es?"

Porque prácticamente todos marcarían **URGENTE**.

Queremos que el sistema determine:

¿Qué tan importante es esta solicitud para la organización?

Y para eso podemos utilizar:

Variable 1 — Cargo

Por ejemplo:

Cargo	Peso
Dirección / Director	Muy alto
Jefatura	Alto
Coordinación	Medio-alto
Profesional	Medio
Administrativo	Normal
Voluntario	Normal

No significa que el director tenga "más derecho" al soporte.

Significa que el **impacto organizacional** potencial de su indisponibilidad puede ser **mayor**.

Variable 2 — Incidencia

Por ejemplo:

Incidencia	Impacto
Sistema institucional caído	Crítico
Sin acceso al correo	Alto
Computador no funciona	Alto
Internet	Medio/alto
Impresora	Medio
Consulta	Bajo

Solicitud de configuración

Bajo

Variable 3 — Descripción

Aquí podemos incorporar posteriormente un clasificador.

Por ejemplo:

"No puedo ingresar a mi correo."

versus

"No puedo ingresar al correo institucional y tengo reunión con el directorio en 15 minutos."

Son técnicamente similares, pero **organizacionalmente no son iguales**.

3. Entonces el sistema calcula

Podríamos tener inicialmente algo tan simple como:

Prioridad = Cargo + Incidencia + Impacto detectado

Y generar:

 **P1 — CRÍTICA**

Atención inmediata.

 **P2 — ALTA**

Atención prioritaria.

 **P3 — NORMAL**

Atención estándar.

 **P4 — BAJA**

Atención programable.

Pero haría una distinción importante

No dejaría que el **cargo por sí solo** pueda convertir una incidencia en crítica.

Por ejemplo:

Director → "Necesito cambiar la firma de mi correo."

No debería necesariamente ser P1.

Mientras que:

Administrativo → "El sistema de remuneraciones está caído para toda la oficina."

podría ser P1.

Por eso el cálculo debe ponderar:

Cargo + naturaleza de la incidencia + alcance + contexto.

Esto evita crear un sistema donde simplemente:

"Director = urgente."

Y eso es importante tanto operacional como políticamente dentro de una organización.

4. Backend del técnico

Aquí cambia completamente la experiencia.

El técnico entra y ve:

Mesa de Ayuda TI

RESUMEN

Abiertos: 12

En proceso: 4

Esperando usuario: 3

Escalados: 1

Vencidos: 2

Y debajo:

Ticket	Usuario	Incidencia	Prioridad	Tiempo
#125	Director	Correo	 P1	00:18
#124	Ana	Notebook	 P2	01:42
#123	Pedro	Impresora	 P3	04:16
#122	María	Consulta	 P4	01:31

5. El timer es fundamental

Me gusta mucho esta idea.

Cada ticket debería tener **un reloj propio**.

Por ejemplo:

TK-00125

 P1

Tiempo transcurrido

01:24:36

Pero iría un paso más allá.

Tendría:

Tiempo total

Desde creación hasta cierre.

Tiempo de primera respuesta

Desde creación hasta que TI comienza a gestionarlo.

Tiempo efectivo de resolución

Descontando períodos en que está:

ESPERANDO USUARIO

Esto permitirá medir realmente el desempeño del soporte.

6. Y aquí aparecen los KPI

Esto convierte el sistema en algo mucho más profesional.

El backend puede mostrar:

KPI DEL SOPORTE

Tickets recibidos

147

Tickets resueltos

132

Pendientes

15

Tiempo promedio de primera respuesta

18 min

Tiempo promedio de resolución

3h 42m

% resueltos dentro del SLA

94%

Tickets vencidos

3

Incidencia más frecuente

Correo electrónico

Área con mayor cantidad de solicitudes

Administración

7. Incluso podemos medir algo más interesante

Carga de soporte

Por ejemplo:

147 tickets / mes

Pero:

Categoría	Tickets
Correo	42
Accesos	31
Computadores	27
Software	21
Internet	15
Otros	11

Entonces la organización puede descubrir:

"Tenemos un problema recurrente de accesos."

Eso ya no es solamente soporte.

Es información para tomar decisiones.

8. Frontend del usuario

También coincido contigo en que **no necesitamos construir un segundo sistema complejo para el usuario.**

El usuario debería tener algo como:

Mis solicitudes

Ticket	Solicitud	Estado	Actualizado
#125	Problema correo	● En proceso	Hoy 10:21
#118	Acceso sistema	● Resuelto	Ayer

Y al entrar:

TK-125

Problema: No puedo acceder al correo.

Estado:

- ☒ Solicitud recibida
- ☒ En proceso
- ☐ Esperando usuario
- ☐ Resuelto

Y nada más.

9. Notificación

Aquí tenemos dos caminos.

Primera etapa

Email

Es barato, sencillo y universal.

Cuando ocurre algo:

- Ticket creado
- Ticket recibido
- Técnico trabajando
- Información requerida
- Ticket resuelto

se envía automáticamente.

Segunda etapa

WhatsApp

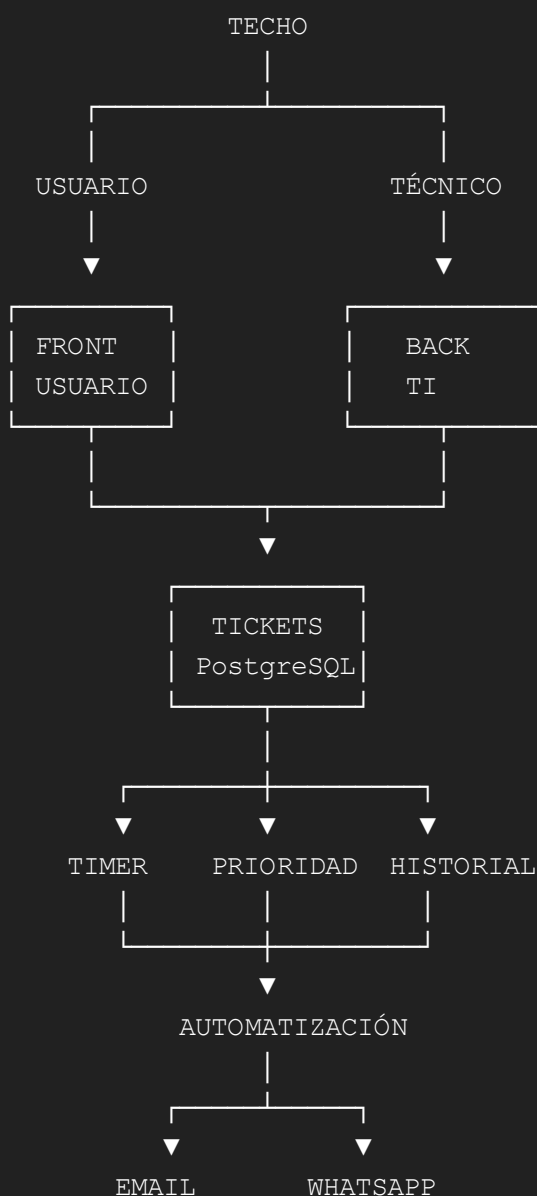
Esto podría ser bastante bueno para TECHO, pero lo dejaría como una segunda fase porque implica integración y costos asociados.

Podríamos incluso permitir:

- Email + WhatsApp**

sin modificar el sistema central.

10. La arquitectura queda extremadamente limpia



Y ahora sí veo claramente el producto

No lo llamaría simplemente "**sistema de tickets**".

La propuesta sería:

Mesa de Ayuda TI inteligente y simplificada

Un sistema que recibe, prioriza, controla y mide automáticamente las solicitudes de soporte, permitiendo que una sola persona gestione eficientemente el soporte tecnológico de toda la organización.

Y el principio de diseño sería:

Para el usuario: 6 pasos.

Para el técnico: toda la información que necesita.

Para la organización: métricas y control.

Eso es exactamente lo que buscaría construir.

Y hay una decisión técnica que tomaría desde el principio

No metería IA generativa todavía.

La prioridad automática puede comenzar perfectamente con **reglas ponderadas**:

Cargo × Incidencia × Impacto

y después, cuando tengamos suficientes tickets históricos, podemos reemplazar o complementar esa lógica con un modelo de clasificación.

Eso hace que el MVP sea **rápido de construir, barato de operar y fácil de explicar a TECHO**, pero deja preparada la plataforma para evolucionar.

En términos de producto, estamos hablando de algo que debería poder convertirse en un **SaaS replicable para organizaciones de 50–500 personas**, no solamente en un desarrollo hecho a medida para TECHO.

la plataforma será un SaaS (en futuro próximo), que permita el desarrollo y mejora a si misma, siendo un asistente de mejoras para el área. si bien en un comienzo será un sistema muy simple y sencillo, en donde existirá tres tipos de usuario (dependiendo el perfil del cargo), en donde el usuario administrativo u operacional (nivel 3) y los voluntarios (nivel 4), ingresará a la plataforma, con un resumen de su cargo (nombre, cargo, departamento, jefatura directa, sucursal y/o local, con el fin de alimentación de data para un futuro), podrá ingresar su requerimiento (propuesta o incidencia), ver el estado de la

[Ver más](#)

Sí. Ahora **DeskWork ya no es simplemente un sistema de tickets**. Lo que acabas de definir cambia el alcance del producto de manera importante, pero sin contradecir la idea inicial de mantenerlo simple.

La clave está en separar dos conceptos:

El MVP es una mesa de ayuda muy sencilla.

DeskWork como SaaS es una plataforma de gestión, inteligencia y evolución del área de soporte.

Y eso me parece una dirección mucho más potente.

DeskWork: concepto central

Yo lo definiría así:

DeskWork es una plataforma SaaS de gestión inteligente de soporte interno que transforma las solicitudes de usuarios en información operativa, métricas de desempeño, conocimiento organizacional y oportunidades de mejora continua.

El objetivo final no es simplemente:

"Resolver computadores."

Es convertir el área TI en un **área de servicio medible, gestionable y capaz de demostrar su aporte a la organización.**

1. La estructura de usuarios que propones está muy bien

Yo mantendría los cinco niveles:

Nivel	Perfil	Función principal
0	Técnico TI	Ejecuta y resuelve
1	Gerente / Director	Visión institucional
2	Jefatura / Supervisor	Gestión de su equipo/área
3	Administrativo / Operacional	Solicita y consulta
4	Voluntario	Solicita y consulta

Y hay una cosa importante:

Nivel jerárquico ≠ permiso técnico.

La plataforma debe utilizar el nivel para determinar **qué información puede ver y qué acciones puede ejecutar**.

2. Nivel 4 — Voluntario

Debe ser extremadamente simple.

Su perfil

- Nombre
- Cargo/rol
- Departamento/área
- Jefatura
- Sucursal/local
- Institución

Esto parece información administrativa, pero en realidad estás creando una **estructura de datos organizacional**.

Y eso será muy valioso después.

Puede:

- crear incidencia;
- crear propuesta;
- consultar solicitudes;
- revisar estado;
- ver tiempo estimado;
- revisar últimos 3–5 requerimientos;
- recibir notificaciones.

Nada más.

3. Nivel 3 — Administrativo / Operacional

Mismo concepto, pero con un perfil organizacional más completo.

Dashboard

Hola, María

Administración

Sucursal Santiago

Jefatura: Juan Pérez

Mis solicitudes

#00152

Problema correo

 En proceso

Respuesta estimada: 1h 30m

#00148

Acceso plataforma

 Resuelto

#00143

Configuración notebook

 Cerrado

Y:

 Nueva solicitud

4. Nivel 2 — Jefaturas / Supervisores

Acá empieza a aparecer realmente el valor empresarial.

El jefe no solamente ve sus propios tickets.

Ve:

 "¿Qué está pasando con mi equipo?"

Dashboard

Mi equipo

12 personas

Solicitudes abiertas: 5

En proceso: 3

Esperando usuario: 1

Resueltas: 17

Y puede entrar:

María Pérez

3 solicitudes

Pedro González

1 solicitud

Juan Soto

5 solicitudes

etc.

5. Pero tu idea de "elevar solicitudes" es especialmente buena

Una jefatura podría decir:

"Necesito que TI revise este problema que afecta a mi equipo."

Entonces podría:

Elevar solicitud existente

o

Crear solicitud para un colaborador

o incluso:

Solicitud departamental

Por ejemplo:

"Necesitamos capacitación en Microsoft 365 para todo el departamento."

Esto ya no es un ticket individual.

Es una **solicitud de servicio/proyecto**.

Y eso nos lleva a una distinción que considero importante para DeskWork:

No todo debería llamarse "ticket".

Podríamos tener:

Incidencia

Problema que requiere solución.

Solicitud

Necesidad de un servicio.

Propuesta

Idea de mejora.

Proyecto

Trabajo de mayor alcance.

Reunión

Solicitud de coordinación.

Eso permite que DeskWork evolucione más allá del HelpDesk.

6. Nivel 1 — Gerentes / Directores

Aquí cambia completamente la perspectiva.

No necesitan ver:

"Juan tiene problema con Outlook."

Necesitan ver:

"¿Cómo está funcionando el servicio tecnológico de la organización?"

Por ejemplo:

DASHBOARD INSTITUCIONAL

Solicitudes

347

Resueltas

312

Pendientes

35

SLA

94,2%

Tiempo promedio resolución

3h 17m

Solicitudes críticas

4

Propuestas de mejora

12

Distribución

Por área

Administración — 28%

Operaciones — 21%

Finanzas — 17%

Dirección — 8%

Otros — 26%

Por tipo

Correo — 31%

Accesos — 24%

Hardware — 18%

Software — 14%

Conectividad — 9%

Otros — 4%

Eso transforma completamente la conversación con la dirección.

Ya no es:

"TI está ocupado."

Sino:

"TI recibió 347 solicitudes, resolvió 312, el 31% correspondió a problemas de correo y tenemos una concentración anormal de solicitudes de acceso."

Eso es **gestión**.

7. Nivel 0 — Técnico TI

Este es el corazón operacional.

El técnico debería entrar y ver:

MI BANDEJA

CRÍTICOS

2

ALTOS

5

NORMALES

11

BAJOS

4

Cada solicitud:

TK-00152

Usuario: Director X

Departamento: Dirección

Cargo: Director

Sucursal: Santiago

Incidencia: Correo

Descripción: ...

Prioridad calculada

 P1

SLA

02:00:00

Timer

00:31:42

Estado

8. El sistema de prioridad que propusiste debe convertirse en un motor

Esto será una de las piezas centrales de DeskWork.

Inicialmente:

```
CARGO
  +
TIPO DE SOLICITUD
  +
DESCRIPCIÓN
  +
ALCANCE
  ↓
MOTOR DE PRIORIDAD
  ↓
P1 / P2 / P3 / P4
```

Y posteriormente:

```
MOTOR DE PRIORIDAD
  +
HISTORIAL
  +
PATRONES
  +
IA
  ↓
PRIORIDAD DINÁMICA
```

Así DeskWork **aprende de la operación**.

9. Y aquí entra la IA

Esto es donde creo que tu concepto empieza a diferenciarse de un HelpDesk convencional.

No pondría una IA simplemente como:

"Chatbot que responde preguntas."

La convertiría en:

DeskWork AI Assistant

Un asistente interno del área.

Puede ayudar al técnico a:

Antes de resolver

Analizar:

Usuario + cargo + incidencia + descripción + historial.

Y sugerir:

Posibles causas

Procedimiento recomendado

Tickets similares

Soluciones anteriores

Durante la resolución

El técnico puede preguntarle:

"¿Cómo resolvimos anteriormente este problema?"

Y la IA busca en el historial.

Después de resolver

La IA puede generar:

Resumen de resolución

Causa

Solución

Tiempo empleado

Conocimiento reutilizable

Esto alimenta automáticamente la **base de conocimiento de DeskWork**.

10. Entonces DeskWork empieza a aprender

Este concepto me parece fundamental.

Cada ticket genera:

Datos



Conocimiento



Patrones



Recomendaciones



Mejoras



Nuevos procedimientos



Menos tickets repetitivos

Ese ciclo es el verdadero diferencial.

11. La IA también puede detectar problemas que nadie está mirando

Por ejemplo:

Durante tres meses:

Accesos: 24 → 31 → 47

DeskWork puede decir:

⚠ Se detecta incremento sostenido de solicitudes relacionadas con accesos.

Y sugerir:

"Evaluar automatización del proceso de creación de usuarios."

Eso ya no es soporte reactivo.

Es **soporte preventivo**.

12. Y después entra Project Management

Esto también tiene mucho sentido con lo que planteas.

Supongamos que aparecen 50 solicitudes sobre:

"Problemas de WiFi."

DeskWork detecta el patrón.

Puede generarse:

PROYECTO

Mejora de conectividad — Sucursal Santiago

Responsable: TI

Inicio: 01/09/2026

Objetivo:

Reducir incidencias de conectividad en un 50%.

Avance

 80%

Tickets asociados

47

Resultado

Incidencias reducidas:

31%

Ahora TI no solamente resolvió tickets.

Ejecutó una mejora medible.

13. Esto cambia la función del departamento

Y creo que esta es exactamente la visión que estás buscando.

TI tradicional

| "Se cayó el computador."

TI:

| "Voy."

TI gestionado por DeskWork

| 47 incidencias similares.

DeskWork:

| "Existe un patrón."

TI:

| "Analizamos causa."

DeskWork:

| "Propongo proyecto."

TI:

| "Implementamos solución."

DeskWork:

| "Medimos resultado."

Dirección:

| "La incidencia disminuyó 62%."

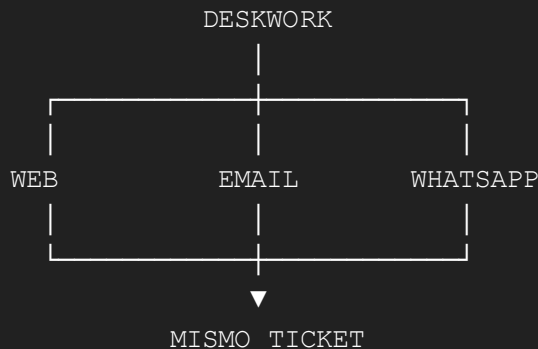
Eso es un **departamento de soporte técnico profesionalizado**.

14. WhatsApp

También lo incorporaría, pero con una regla fundamental:

WhatsApp no crea un sistema paralelo.

Debe ser solamente **otra puerta de entrada a DeskWork**.



Por ejemplo:

Usuario:

"Hola, no puedo entrar al correo."

DeskWork identifica:

+56 XXXXXXXX → usuario → cargo → área

y crea:

TK-00153

Luego:

"Tu solicitud #00153 fue recibida."

Y posteriormente:

● Tu solicitud está en proceso.

Y al terminar:

● Tu solicitud #00153 fue resuelta.

El usuario no tiene que entrar a la plataforma para todo.

15. Y esto nos lleva al SaaS

Aquí creo que debemos hacer una distinción arquitectónica desde el principio.

TECHO será:

Tenant

TECHO Chile

Y dentro:

```
TECHO
|
├── Usuarios
├── Departamentos
├── Sucursales
├── Cargos
├── Jefaturas
├── Técnicos
├── Tickets
├── Proyectos
├── Métricas
└── Conocimiento
```

Después puedes tener:

```
DESKWORK SaaS
|
├── TECHO
├── Empresa B
├── Fundación C
├── Colegio D
├── ONG E
└── Empresa F
```

Cada organización tiene su propio entorno.

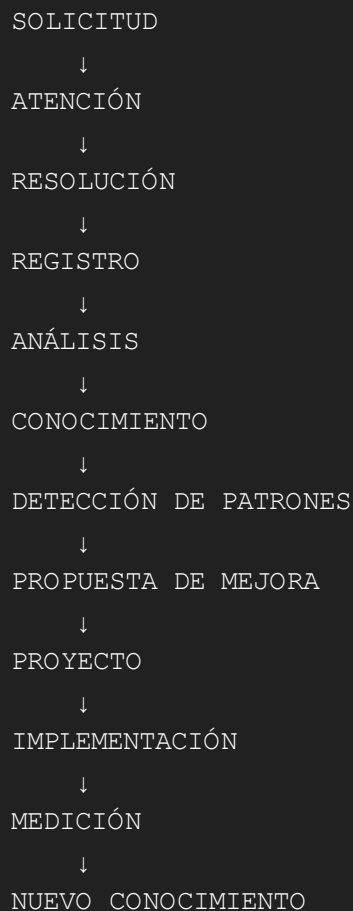
16. Y el gran objetivo de DeskWork

Aquí está la parte que yo pondría como **visión de producto**:

DeskWork no solo administra solicitudes. Aprende de ellas para ayudar a la

organización a mejorar la forma en que trabaja.

Por eso la plataforma debería tener un ciclo:



Ese ciclo es DeskWork.

Cómo lo estructuraría como producto

No intentaría construir todo esto ahora.

DESKWORK 0.1 — MVP TECHO

Usuarios

- Nivel 4
- Nivel 3
- Nivel 2
- Nivel 1
- Nivel 0

Core

- autenticación;
- perfiles;

- organización;
- tickets;
- incidencias;
- solicitudes;
- prioridad automática;
- estados;
- timer;
- notificaciones;
- dashboard técnico;
- dashboard ejecutivo;
- historial.

DESKWORK 0.2

- WhatsApp;
- SLA;
- KPI avanzados;
- reportes;
- base de conocimiento;
- inventario.

DESKWORK 0.3

- IA Assistant;
- análisis de patrones;
- recomendaciones;
- generación automática de reportes;
- clasificación inteligente.

DESKWORK 1.0

- proyectos;
- gestión de mejoras;
- medición de impacto;
- inteligencia operacional;
- benchmarking;
- aprendizaje organizacional.

Y recién ahí tenemos el **SaaS plenamente comercializable**.

Y hay algo que mantendría como principio de diseño desde el primer día

Aunque el primer cliente sea TECHO y el MVP sea pequeño, **la base de datos y la arquitectura deben nacer multi-tenant.**

No necesitamos implementar todas las funciones SaaS ahora, pero sí evitar construir algo que después haya que botar para convertirlo en SaaS.

Así conseguimos exactamente lo que estás buscando:

MVP pequeño → operación real → datos → aprendizaje → mejoras → SaaS.

Y el primer cliente no sería solamente un cliente: sería el **entorno real de validación de DeskWork.**

con esto, genera un prompt para el desarrollo de una presentación, para poder tener una propuesta estética, simple y practica que mostrar (dossier), explayando la explicación al cliente la propuesta de trabajo.

Sí. En este caso conviene que el prompt **no le pida a la IA solamente “hacer una presentación bonita”**, sino que le entregue el concepto de DeskWork, el problema que resuelve, el MVP, los perfiles de usuario, la evolución SaaS y, sobre todo, **cómo debe venderse la propuesta a TECHO.**

La presentación debería funcionar como **dossier comercial + explicación ejecutiva**, no como manual técnico.

Te propongo este prompt maestro:

Editar

PROMPT MAESTRO — DESARROLLO DE PRESENTACIÓN COMERCIAL DESKWORK

1. ROL

Actúa como un equipo multidisciplinario compuesto por:

- Director de producto SaaS.

- Consultor de transformación digital.
- Especialista en UX/UI.
- Diseñador de presentaciones corporativas.
- Consultor de gestión TI.
- Especialista en automatización e inteligencia artificial.
- Director comercial B2B.

Tu objetivo es desarrollar una **presentación/dossier comercial de DeskWork**, una plataforma SaaS de gestión inteligente de soporte interno, utilizando como primer caso de implementación a **Fundación TECHO-Chile**.

La presentación será utilizada para mostrar la propuesta a un potencial mandante y explicar de manera clara:

1. El problema actual.
2. La solución propuesta.
3. Cómo funcionará DeskWork.
4. Qué verá cada tipo de usuario.
5. Cómo funcionará el área técnica.
6. Cómo se medirán los resultados.
7. Cómo se incorporará IA.
8. Cómo evolucionará el sistema.
9. Cómo será la implementación inicial.
10. Cómo puede evolucionar posteriormente hacia un SaaS.

La presentación debe ser **estética, simple, profesional, moderna y práctica**.

NO debe parecer una presentación de una empresa de software genérica.

Debe parecer un **producto real que está siendo propuesto para resolver una necesidad concreta**.

2. CONTEXTO DEL PROYECTO

El proyecto se denomina:

DESKWORK

Concepto:

DeskWork es una plataforma de gestión inteligente de soporte interno que transforma las solicitudes de usuarios en información operativa, métricas de desempeño, conocimiento organizacional y oportunidades de mejora continua.

El primer caso de aplicación será Fundación TECHO-Chile.

La organización considerada para el MVP tiene aproximadamente:

- 185 trabajadores.
- Una cantidad adicional importante de voluntarios.
- Un área de soporte TI actualmente gestionada por una sola persona.

El problema principal no es necesariamente la falta de conocimientos técnicos.

El problema es que una sola persona debe atender las solicitudes de toda la organización y necesita una herramienta que le permita:

- recibir solicitudes ordenadamente;
- priorizarlas;
- saber qué está pendiente;
- medir tiempos;
- mantener historial;
- demostrar resultados;
- identificar problemas recurrentes;
- generar reportes;
- mejorar progresivamente el servicio.

La solución debe comenzar siendo **muy simple**.

No desarrollar inicialmente un sistema ITSM complejo.

3. IDEA CENTRAL DE LA PROPUESTA

La presentación debe transmitir una idea:

"Transformar el soporte TI desde una atención reactiva y difícil de medir, hacia un servicio organizado, trazable, medible y

orientado a la mejora continua."

No presentar DeskWork como:

"un sistema de tickets."

Presentarlo como:

Una plataforma para gestionar y hacer evolucionar el servicio interno de soporte.

El ticket es solamente la unidad básica de información.

4. PRINCIPIO FUNDAMENTAL DE UX

El usuario final debe hacer muy poco.

La experiencia debe reducirse prácticamente a:

1. Entrar

2. Identificarse

El sistema reconoce su correo institucional y recupera:

- nombre;
- cargo;
- departamento;
- jefatura;
- sucursal/local;
- organización.

3. Indicar incidencia o solicitud

Categorías iniciales:

- Computador.
- Correo.
- Internet/conectividad.
- Impresora.
- Telefonía.
- Accesos/permisos.

- Software/aplicaciones.
- Cuenta/usuario.
- Otro.

4. Describir

Campo libre para explicar la necesidad o problema.

5. Adjuntar imagen

Opcional.

6. Enviar

Eso es todo.

El usuario NO debe definir:

- prioridad;
- SLA;
- técnico;
- fecha de resolución;
- estado técnico.

Todo eso debe ser gestionado por DeskWork.

5. PRIORIDAD INTELIGENTE

Una de las características diferenciadoras de DeskWork será que el usuario **no determina la prioridad**.

El sistema calculará la prioridad considerando inicialmente:

- cargo;
- incidencia;
- descripción;
- alcance;
- impacto potencial.

Ejemplo conceptual:

Un Director solicitando una modificación menor de correo no necesariamente es una incidencia crítica.

Mientras que un usuario administrativo reportando una caída de un sistema utilizado por todo un departamento puede tener una prioridad mucho mayor.

Por lo tanto, mostrar visualmente:

CARGO

+

TIPO DE INCIDENCIA

+

DESCRIPCIÓN

+

IMPACTO

↓

MOTOR DE PRIORIZACIÓN

↓

P1 / P2 / P3 / P4

Presentar esto como una **priorización objetiva y basada en contexto**, no como un sistema que privilegia arbitrariamente a las personas de mayor jerarquía.

En el MVP puede comenzar mediante reglas ponderadas.

Posteriormente puede evolucionar mediante IA y aprendizaje histórico.

6. NIVELES DE USUARIO

Mostrar claramente los cinco niveles.

NIVEL 4 — VOLUNTARIO

Puede:

- ingresar solicitudes;
- realizar propuestas;
- consultar estado;
- consultar tiempo estimado;
- revisar sus últimos 3–5 requerimientos;
- recibir notificaciones.

NIVEL 3 — ADMINISTRATIVO / OPERACIONAL

Tiene las mismas funciones del nivel 4, además de un perfil organizacional más completo.

Puede consultar:

- nombre;
- cargo;
- departamento;
- jefatura;
- sucursal/local;
- historial de solicitudes.

NIVEL 2 — JEFATURA / SUPERVISOR

Además de sus propias solicitudes puede:

- visualizar solicitudes de sus dependientes;
- elevar solicitudes;
- realizar solicitudes para usuarios;
- visualizar solicitudes de su departamento;
- hacer seguimiento;
- solicitar reuniones;
- solicitar seguimiento;
- acceder a información de su área.

NIVEL 1 — GERENTE / DIRECTOR

Tiene visión institucional.

Puede:

- visualizar requerimientos globales;
- consultar estados;
- visualizar estadísticas;
- consultar métricas;
- revisar situaciones relevantes;
- revisar solicitudes institucionales;
- acceder a reportes;
- realizar determinadas acciones administrativas según configuración del mandante.

NIVEL 0 — SOPORTE TI

Es el usuario operativo principal.

Tiene acceso al backend de DeskWork.

Puede:

- recibir solicitudes;
- visualizar información del solicitante;
- revisar prioridad;
- gestionar estados;
- trabajar tickets;
- registrar acciones;
- solicitar información;
- escalar;
- resolver;
- cerrar;
- consultar historial;
- utilizar la asistencia de IA;
- generar reportes.

7. FRONTEND DEL USUARIO

Mostrar una maqueta conceptual de una pantalla.

Debe ser limpia y minimalista.

Ejemplo:

Hola, María

Administración

Sucursal Santiago

Jefatura: Juan Pérez

Mis solicitudes

#00152 — Problema correo

 En proceso

Tiempo estimado: 1h 30m

#00148 — Acceso plataforma

 Resuelto

#00143 — Configuración notebook

 Cerrado

NUEVA SOLICITUD

[Crear solicitud]

No llenar la pantalla de información.

La experiencia debe ser intuitiva.

8. BACKEND DEL TÉCNICO

Crear una maqueta conceptual del panel.

Debe mostrar:

DESKWORK — SOPORTE TI

Abiertos: 12

En proceso: 4

Esperando usuario: 3

Escalados: 1

Vencidos: 2

Después mostrar una bandeja de tickets:

Ticket	Usuario	Incidencia	Prioridad	Tiempo
#125	Director	Correo	P1	00:18
#124	Ana	Notebook	P2	01:42

Destacar visualmente el TIMER.

Cada solicitud debe tener un contador de tiempo.

9. ESTADOS

Usar una lógica sencilla:

1. ABIERTO
2. EN PROCESO
3. ESPERANDO USUARIO
4. ESCALADO
5. RESUELTO
6. CERRADO

Mostrar visualmente mediante un tablero Kanban.

No agregar estados innecesarios.

10. TIMER Y KPI

El timer es una característica central.

DeskWork debe medir:

- tiempo de recepción;
- tiempo de primera respuesta;
- tiempo en proceso;
- tiempo esperando usuario;
- tiempo de resolución;
- tiempo total;
- cumplimiento de SLA.

Mostrar después un dashboard:

KPI SOPORTE TI

Solicitudes recibidas

147

Solicitudes resueltas

132

Pendientes

15

Tiempo promedio de primera respuesta

18 min

Tiempo promedio de resolución

3h 42m

SLA cumplido

94%

Tickets vencidos

3

11. DASHBOARD EJECUTIVO

Mostrar la diferencia entre el panel técnico y el panel ejecutivo.

El ejecutivo NO necesita ver el detalle de cada ticket.

Necesita saber:

¿Cómo está funcionando el servicio?

Mostrar:

SOLICITUDES

347

RESUELTAS

312

PENDIENTES

35

SLA

94,2%

TIEMPO PROMEDIO

3h 17m

INCIDENCIA MÁS FRECUENTE

Correo

Después:

- solicitudes por área;
- solicitudes por categoría;
- solicitudes por prioridad;
- evolución mensual;
- tickets críticos;
- cumplimiento de SLA;
- principales problemas recurrentes.

12. WHATSAPP

Mostrar que WhatsApp será otro canal de entrada, pero NO otro sistema.

Concepto:

WEB

+

WHATSAPP

+

EMAIL

↓

DESKWORK

↓

MISMO TICKET

Un usuario puede enviar:

"Hola, no puedo acceder al correo."

DeskWork identifica al usuario mediante su número vinculado.

Genera automáticamente:

TK-00153

Y posteriormente puede enviar:

Hemos recibido tu solicitud #00153.

Tu solicitud está en proceso.

Tu solicitud ha sido resuelta.

El sistema central sigue siendo DeskWork.

WhatsApp es solamente una interfaz de acceso.

13. INTELIGENCIA ARTIFICIAL

Presentar la IA como una evolución progresiva.

NO venderla como una inteligencia artificial espectacular desde el primer día.

Mostrar:

DeskWork AI Assistant

Puede ayudar al técnico a:

- analizar solicitudes;
- sugerir prioridad;
- encontrar tickets similares;
- buscar soluciones anteriores;
- sugerir procedimientos;
- resumir conversaciones;
- generar respuestas;
- documentar resoluciones;
- generar reportes;
- detectar patrones.

Posteriormente:

- identificar incidencias recurrentes;
 - detectar problemas preventivamente;
 - recomendar mejoras;
 - generar conocimiento;
 - apoyar proyectos.
-

14. APRENDIZAJE DEL SISTEMA

Esta debe ser una de las diapositivas conceptualmente más importantes.

Mostrar:

SOLICITUD

↓

RESOLUCIÓN

↓

DATOS

↓

CONOCIMIENTO

↓

PATRONES

↓

MEJORA

↓

PROYECTO

↓

RESULTADO

↓

NUEVO CONOCIMIENTO

La idea es:

Cada solicitud resuelta aumenta el conocimiento de DeskWork.

Por lo tanto, la plataforma mejora progresivamente con el uso.

15. DE SOPORTE REACTIVO A SOPORTE ESTRATÉGICO

Crear una diapositiva de contraste.

ANTES

"Se rompió un computador."

"Hay problemas de correo."

"Alguien necesita ayuda."

"TI está ocupado."

Difícil medir.

Difícil demostrar resultados.

Difícil identificar patrones.

CON DESKWORK

"147 solicitudes recibidas."

"94% dentro de SLA."

"31% relacionadas con correo."

"Existe un problema recurrente de accesos."

"Se propone una mejora."

"El proyecto redujo las incidencias en 42%."

El objetivo es demostrar cómo DeskWork permite que TI deje de ser percibido únicamente como:

"La persona que arregla los computadores."

y pase a ser:

Un área de servicio, gestión, información y mejora continua.

16. GESTIÓN DE PROYECTOS Y MEJORAS

Explicar que las incidencias repetitivas pueden transformarse en iniciativas.

Ejemplo:

47 tickets de conectividad.



DeskWork detecta patrón.



Se crea proyecto:

Mejora de conectividad — Sucursal Santiago



Objetivo:

Reducir incidencias de conectividad en 50%.



Seguimiento:



Resultado:

Incidencias reducidas 62%.

Esto demuestra cómo DeskWork permite pasar de:

ticket → análisis → proyecto → mejora → medición.

17. EVOLUCIÓN DEL PRODUCTO

Mostrar un roadmap simple.

ETAPA 1 — MVP

Mesa de ayuda.

- Usuarios.
- Solicitudes.
- Incidencias.
- Estados.
- Timer.
- Notificaciones.
- Dashboard.
- KPI.

ETAPA 2 — GESTIÓN

- WhatsApp.
- SLA.
- Base de conocimiento.
- Inventario.
- Reportes avanzados.

ETAPA 3 — INTELIGENCIA

- DeskWork AI.
- Análisis de patrones.
- Recomendaciones.
- Automatización.
- Reportes inteligentes.

ETAPA 4 — MEJORA CONTINUA

- Gestión de proyectos.
- Iniciativas.
- Medición de impacto.
- Benchmarking.
- Inteligencia operacional.

ETAPA 5 — SaaS

DeskWork disponible para múltiples organizaciones.

Cada organización tendrá:

- usuarios;

- estructura;
 - departamentos;
 - técnicos;
 - solicitudes;
 - métricas;
 - conocimiento;
 - configuración independiente.
-

18. IMPLEMENTACIÓN PROPUESTA PARA TECHO

Presentar una implementación progresiva.

FASE 1 — LEVANTAMIENTO

Comprender:

- estructura;
- usuarios;
- departamentos;
- cargos;
- flujo actual;
- categorías;
- necesidades del área TI.

FASE 2 — CONFIGURACIÓN

Configurar DeskWork para TECHO.

FASE 3 — MVP

Implementar:

- usuarios;
- solicitudes;
- prioridades;
- estados;
- timer;
- dashboard;
- notificaciones.

FASE 4 — PILOTO

Comenzar con un grupo controlado.

Medir.

Ajustar.

FASE 5 — IMPLEMENTACIÓN

Extender al resto de trabajadores.

FASE 6 — EVOLUCIÓN

Agregar funcionalidades según necesidades reales.

19. MODELO COMERCIAL

No presentar todavía precios concretos salvo que se proporcionen.

Explicar el modelo:

IMPLEMENTACIÓN

Pago único.

Incluye:

- levantamiento;
- diseño;
- configuración;
- desarrollo;
- puesta en marcha;
- capacitación.

EVOLUCIÓN / MANTENIMIENTO

Servicio mensual opcional.

Incluye según plan:

- mantenimiento;
- correcciones;
- ajustes;
- mejoras;
- soporte;

- evolución de funcionalidades.

DESARROLLOS ESPECIALES

Nuevas funcionalidades de mayor alcance pueden ser cotizadas separadamente.

El concepto comercial debe ser:

La organización paga por implementar la solución y posteriormente decide cuánto quiere evolucionarla.

No crear dependencia artificial.

20. ARQUITECTURA

Mostrar únicamente una arquitectura conceptual.

No hacer una diapositiva excesivamente técnica.

Debe verse:

USUARIOS



DESKTOP



MOTOR DE SOLICITUDES



MOTOR DE PRIORIDAD



GESTIÓN



DATOS



MÉTRICAS



IA



MEJORA CONTINUA

La arquitectura tecnológica detallada queda fuera del dossier comercial.

21. SEGURIDAD Y DATOS

Incluir una diapositiva breve.

Considerar:

- autenticación;
- perfiles y permisos;
- separación de información por niveles;
- trazabilidad;
- historial;
- respaldo;
- protección de información;
- arquitectura preparada para múltiples organizaciones.

No realizar afirmaciones legales o certificaciones que no hayan sido definidas.

22. ESTILO VISUAL

La presentación debe tener un diseño:

- moderno;
- tecnológico;
- sobrio;
- institucional;
- limpio;
- minimalista;
- profesional.

Evitar:

- exceso de texto;
- exceso de colores;
- fotografías genéricas de personas usando computadores;
- gráficos decorativos sin información;
- estética excesivamente futurista;

- exceso de iconos;
- lenguaje técnico innecesario.

Preferir:

- diagramas simples;
- interfaces conceptuales;
- dashboards;
- tarjetas KPI;
- flujos;
- timelines;
- mockups;
- jerarquía visual;
- espacios en blanco.

Utilizar una identidad visual propia de DeskWork.

Debe transmitir:

orden + eficiencia + inteligencia + simplicidad.

23. PALETA VISUAL

Proponer una paleta corporativa sobria.

Base:

- blanco;
- negro/grafito;
- gris;
- un color principal de DeskWork;
- colores funcionales para estados:
 - rojo = crítico;
 - naranja = alto;
 - amarillo = normal;
 - verde = resuelto.

No abusar de colores.

24. TIPOGRAFÍA

Utilizar una tipografía moderna y altamente legible.

Preferencias:

- Inter;
- Manrope;
- IBM Plex Sans;
- equivalente moderna.

Títulos grandes.

Poco texto.

Mucho espacio.

25. ESTRUCTURA RECOMENDADA DEL DOSSIER

Crear aproximadamente 14–18 diapositivas.

01

PORTADA

DESKWORK

Gestión inteligente del soporte interno.

Caso inicial:

Fundación TECHO-Chile

02

EL PROBLEMA

03

LA OPORTUNIDAD

04

LA PROPUESTA

05

¿CÓMO FUNCIONA?

06

EXPERIENCIA DEL USUARIO

07

NIVELES DE USUARIO

08

PRIORIZACIÓN INTELIGENTE

09

PANEL DEL TÉCNICO

10

TIMER + KPI

11

DASHBOARD EJECUTIVO

12

WHATSAPP + CANALES

13

DESKWORK AI

14

DE SOPORTE A MEJORA CONTINUA

15

GESTIÓN DE PROYECTOS

16

ROADMAP

17

26. TONO

La presentación debe hablarle a un mandante, no a un programador.

Usar lenguaje:

- claro;
- directo;
- profesional;
- comercial;
- comprensible;
- orientado a resultados.

No utilizar innecesariamente:

- API;
- microservicios;
- PostgreSQL;
- Docker;
- Kubernetes;
- arquitectura distribuida;
- RLS;
- vectores;
- embeddings.

Esos conceptos pueden existir técnicamente, pero no son el argumento comercial.

27. MENSAJE FINAL

Cerrar la presentación con una frase potente:

**DeskWork comienza resolviendo solicitudes.
Evoluciona aprendiendo de ellas.
Y termina ayudando a la organización a mejorar.**

Debajo:

De soporte reactivo → a servicio medible → a mejora continua.

28. REGLA FUNDAMENTAL

No inventes funcionalidades que no estén descritas en este documento.

Si propones una funcionalidad adicional, diferénciala claramente como:

"Evolución futura sugerida".

No presentes funcionalidades futuras como si fueran parte del MVP.

La presentación debe diferenciar visualmente:

HOY / MVP

de

FUTURO / ROADMAP

El resultado debe ser un dossier que permita que un representante de TECHO entienda DeskWork en pocos minutos y que posteriormente el presentador pueda utilizar cada diapositiva para profundizar verbalmente en la propuesta.

No debe parecer que estamos intentando vender una plataforma gigantesca.

Debe transmitir:

"Comenzamos simple, resolvemos un problema concreto y construimos sobre datos reales."

genera un documento maestro explicando el proyecto, sacando a techo como cliente, para que la IA que tome esto no se quede solo con esto, sino que con la matriz del proyecto, los acuerdos, diseño, estructura y todo lo que hemos hablado, evitando así tener que copiar y pegar todo lo que hemos estado hablando.

Listo. Preparé el **Documento Maestro de DeskWork** como una fuente de contexto independiente, pensada precisamente para que puedas entregársela a Claude, Kimi, Minimax, Codex u otra IA **sin tener que copiar nuevamente toda la conversación**.

Incluye:

- visión y propósito del producto;
- problema que resuelve;
- principios de diseño;
- matriz completa de usuarios **Nivel 0–4**;
- experiencia Frontend;
- Backend del técnico;
- motor de prioridad;
- estados;
- timer y KPI;
- dashboards;
- canales Web / Email / WhatsApp;
- IA Assistant;
- conocimiento y aprendizaje;
- mejora continua;
- gestión de proyectos;
- arquitectura conceptual;
- modelo SaaS;
- roadmap;
- MVP;
- modelo comercial;
- qué **no** construir inicialmente;
- criterios de éxito;
- preguntas pendientes;
- reglas para que futuras IA mantengan continuidad y no inventen decisiones.

También **saqué a TECHO como definición del producto**. Queda entendido como el eventual primer caso/piloto, pero DeskWork está documentado como producto independiente y futuro SaaS.

Documento

Descargar DeskWork — Documento Maestro del Proyecto (.docx)

Mi recomendación es que este documento pase a ser el “**Master Context / Single Source of Truth**” de **DeskWork**. A partir de ahora, cualquier IA que trabaje en arquitectura, UX, desarrollo, presentación o modelo comercial debería recibir primero este documento y luego instrucciones específicas de la tarea. 

DeskWork_Documento_Maestro.docx
Documento

ChatGPT puede cometer errores. Se debe comprobar la información importante